FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	**Paolo C. Gorospe**
Section:	**AN43**
Professor:	**Crisola Tan**
Date:	**November 2, 2026**

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF-THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Completed Laboratory Tasks

Part 1 Knowledge base and use case

This classroom comfort and weather assistant recommends cooling and rain precautions. The knowledge base contains five facts in three categories: environment (temperature_c = 32, humidity_pct = 85, is_raining = True), room (is_occupied = True), and equipment (aircon_available = True). Temperature is in degrees Celsius and humidity is a percentage. The Boolean facts record presence or availability. Both reasoning systems use this same input.

Part 2 Rule based reasoning

Four IF–THEN functions represent the rules. An inference engine copies the facts into working memory, evaluates every rule, records its result, and collects all distinct actions. For this input, hot, humid, and rain rules fire; the empty-room rule does not. The three actions are to turn on the aircon, use dry mode, and bring an umbrella.

Part 3 Case based reasoning

Three illustrative past cases store problem features and solutions. The current facts become five Boolean features: hot (> 30 °C), humid (> 70%), raining, occupied, and aircon available. Similarity equals matching features divided by five, with equal weights. Cases 1, 2, and 3 score 80%, 60%, and 20%; ties select the first case. Retrieval chooses Case 1 and reuse returns its cooling advice. Revision adds umbrella advice because the current case is rainy. Retention adds Case 4, which then scores 100%. Retention is in memory for this run; restarting restores the three initial cases.

Run with Python 3: python TA4_Gorospe.py. Enter a revised solution or press Enter to keep it. Use python TA4_Gorospe.py --demo to reproduce the displayed output. No external Python packages are required.

PART 4 COMPARATIVE REFLECTION

1 Which system was easier to implement and why

RBR was easier to implement for this task because each condition could be written directly as a small function. CBR required additional steps for feature conversion, similarity scoring, retrieval, revision, and retention.

2 Which system is more flexible for unstructured problems

CBR is more flexible for less structured problems because a partially similar experience can suggest a solution, although the problem must still be represented by useful features.

3 Which system is better for continuously changing data

CBR is often better when new situations and solutions continually emerge, provided retained solutions are checked and outdated cases are reviewed. If only sensor values change while decision rules stay fixed, RBR remains suitable.

4 Real world applications of both systems

RBR is useful in thermostat control and equipment alarm systems. CBR is useful in help-desk troubleshooting and repair recommendation systems. In this activity, RBR immediately includes umbrella advice, whereas CBR needs revision because its closest past case was not rainy.

Complete Python Code 1 of 4

Knowledge base and rule functions — source lines 1–40

```
"""TA4: KR, RBR and CBR - Paolo C. Gorospe, AN43.
Run: python TA4_Gorospe.py (manual revision)
Or:  python TA4_Gorospe.py --demo (reproducible example)
Uses only the Python standard library. Cases are retained in memory.
"""
from copy import deepcopy
import sys

# Task 1A: five facts, grouped into three categories.
facts = {
    "environment": {"temperature_c": 32, "humidity_pct": 85,
                    "is_raining": True},
    "room": {"is_occupied": True},
    "equipment": {"aircon_available": True},
}


# Task 2A: each function implements one IF-THEN rule.
def rule_hot(f):
    if (f["environment"]["temperature_c"] > 30
            and f["room"]["is_occupied"]
            and f["equipment"]["aircon_available"]):
        return "Turn on the aircon."


def rule_humid(f):
    if (f["environment"]["humidity_pct"] > 70
            and f["room"]["is_occupied"]
            and f["equipment"]["aircon_available"]):
        return "Use the aircon's dry mode."


def rule_rain(f):
    if f["environment"]["is_raining"]:
        return "Bring an umbrella."


def rule_empty(f):
    if not f["room"]["is_occupied"]:
        return "Keep cooling equipment off in the empty room."
```

Complete Python Code 2 of 4

Inference engine and case base — source lines 41–84

```
# Task 2B: check ALL rules against a working-memory snapshot.
def infer(f):
    working_memory = deepcopy(f)
    actions, trace = [], []
    for rule in (rule_hot, rule_humid, rule_rain, rule_empty):
        action = rule(working_memory)
        trace.append((rule.__name__, bool(action)))
        if action and action not in actions:
            actions.append(action)
    return actions, trace


# CBR uses Boolean features so all five comparisons have equal weight.
def features(f):
    return {"hot": f["environment"]["temperature_c"] > 30,
            "humid": f["environment"]["humidity_pct"] > 70,
            "raining": f["environment"]["is_raining"],
            "occupied": f["room"]["is_occupied"],
            "aircon": f["equipment"]["aircon_available"]}


# Task 3A: three illustrative past cases, each with a solution.
case_base = [
    {"id": 1, "problem": {"hot": True, "humid": True,
     "raining": False, "occupied": True, "aircon": True},
     "solution": "Turn on the aircon and use dry mode."},
    {"id": 2, "problem": {"hot": False, "humid": True,
     "raining": True, "occupied": True, "aircon": False},
     "solution": "Use a fan and bring an umbrella."},
    {"id": 3, "problem": {"hot": False, "humid": False,
     "raining": False, "occupied": False, "aircon": True},
     "solution": "Keep cooling equipment off."},
]


# Retrieve: fraction of matching features; ties keep the first case.
def similarity(problem, past):
    if not problem:
        raise ValueError("At least one problem feature is required.")
    matches = sum(k in past and past[k] == v
                  for k, v in problem.items())
    return matches / len(problem)
```

Complete Python Code 3 of 4

The four CBR steps and output — source lines 85–126

```
def retrieve(problem, cases):
    if not cases:
        raise ValueError("The case base is empty.")
    scores = [similarity(problem, c["problem"]) for c in cases]
    best_index = max(range(len(cases)), key=lambda i: scores[i])
    return cases[best_index], scores


# Task 3C: reuse the retrieved solution without changing its case.
def reuse(case):
    return case["solution"]


# Task 3D: a blank response keeps the solution unchanged.
def revise(solution, replacement):
    return replacement.strip() or solution


# Task 3E: retain an independent copy for later retrieval in this run.
def retain(problem, solution, cases):
    new_id = max((c["id"] for c in cases), default=0) + 1
    cases.append({"id": new_id, "problem": deepcopy(problem),
                  "solution": solution})
    return cases[-1]


def main():
    print("TA4 - Paolo C. Gorospe | AN43 | November 2, 2026")
    print("INPUT - Knowledge base (3 categories, 5 facts)")
    for category, values in facts.items():
        print(f"{category}: {values}")
    actions, trace = infer(facts)
    print("\nRBR PROCESS - Check every rule")
    for name, fired in trace:
        print(f"{name}: {'FIRED' if fired else 'not fired'}")
    print("Rule-Based Reasoning Actions:")
    for action in actions:
        print(f"- {action}")
    if not actions:
        print("- No rule matched.")
```

Complete Python Code 4 of 4

CBR execution and retention — source lines 127–156

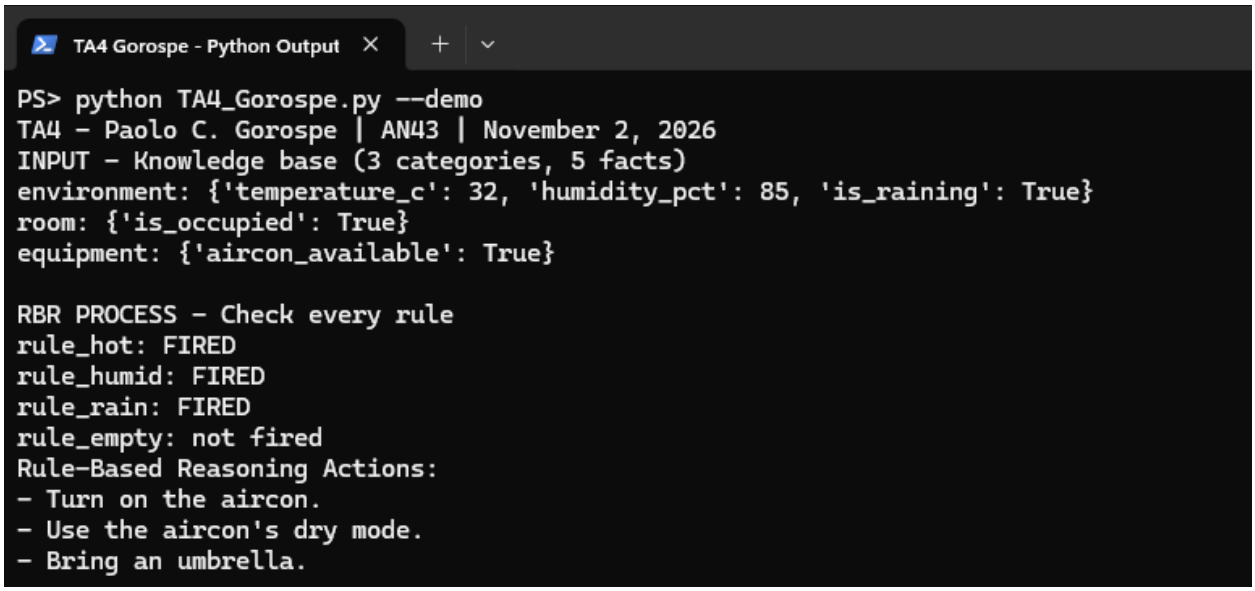
```
    problem = features(facts)
    cases = deepcopy(case_base)
    print("\nCBR INPUT - Features from the same knowledge base")
    print(problem)
    best, scores = retrieve(problem, cases)
    print("RETRIEVE - Matching features / 5")
    for case, score in zip(cases, scores):
        print(f"Case {case['id']}: {score:.0%} similarity")
    proposed = reuse(best)
    print(f"REUSE - Case {best['id']}: {proposed}")
    if "--demo" in sys.argv:
        replacement = proposed + " Bring an umbrella."
        print("REVISE - Demo manual adjustment: add umbrella advice.")
    else:
        replacement = input("REVISE - New solution (Enter to keep): ")
    final = revise(proposed, replacement)
    print(f"Final solution: {final}")
    before = len(cases)
    new_case = retain(problem, final, cases)
    print(f"RETAIN - Cases: {before} -> {len(cases)}")
    print(f"New case ID: {new_case['id']}")
    again, _ = retrieve(problem, cases)
    score = similarity(problem, again["problem"])
    print(f"RETRIEVE AGAIN - Case {again['id']}: {score:.0%} similarity")
    print("Retention lasts for this run; restarting restores 3 cases.")


if __name__ == "__main__":
    main()
```

Program Output

Screenshots from an actual terminal run of python TA4_Gorospe.py --demo. The demo supplies the shown revision; the normal run accepts a manual replacement or an unchanged solution.

Task 2C RBR input process and actions



Part 3 CBR retrieval reuse revision and retention

